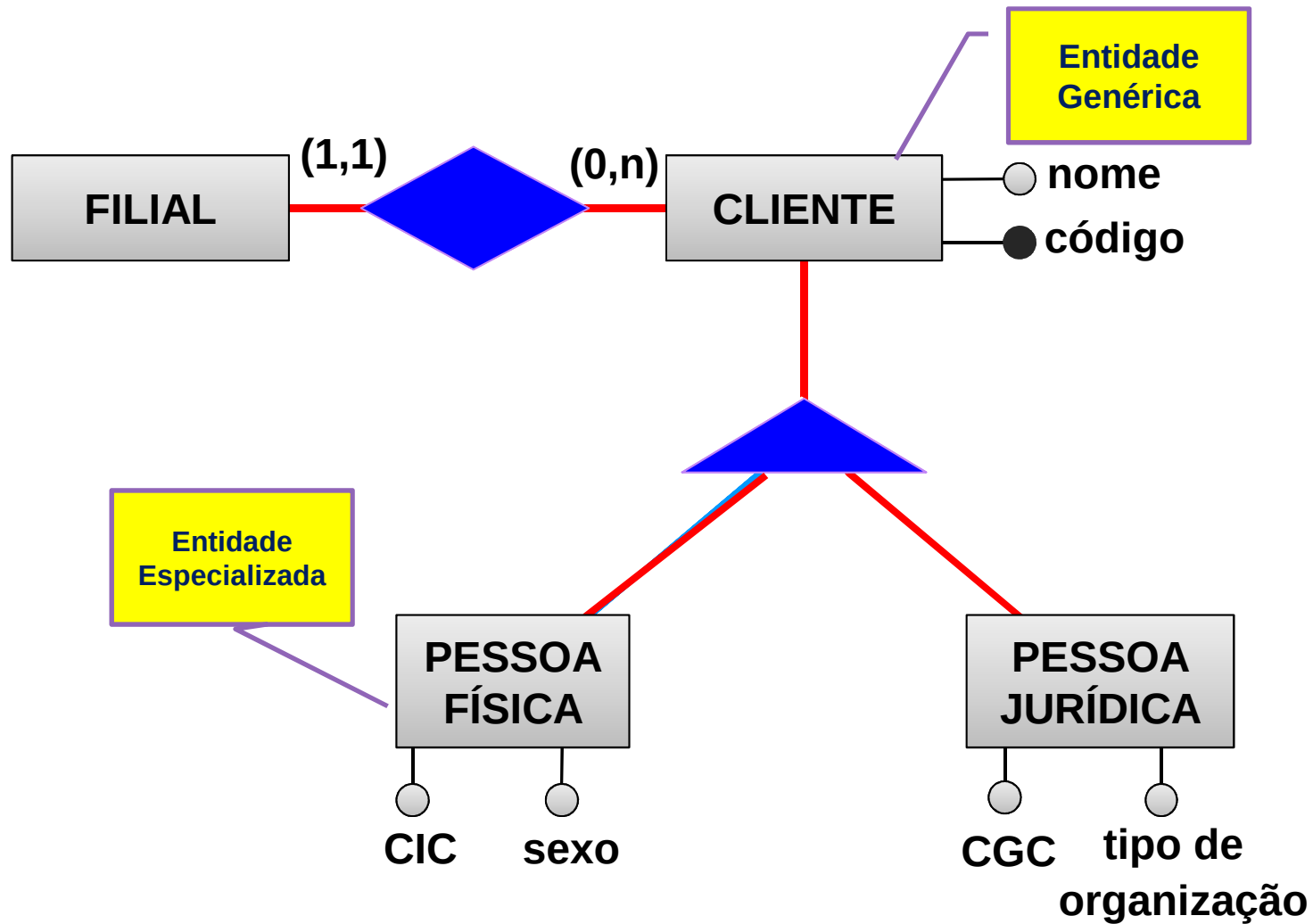


Análise e desenvolvimento de Sistemas

BANCO DE DADOS

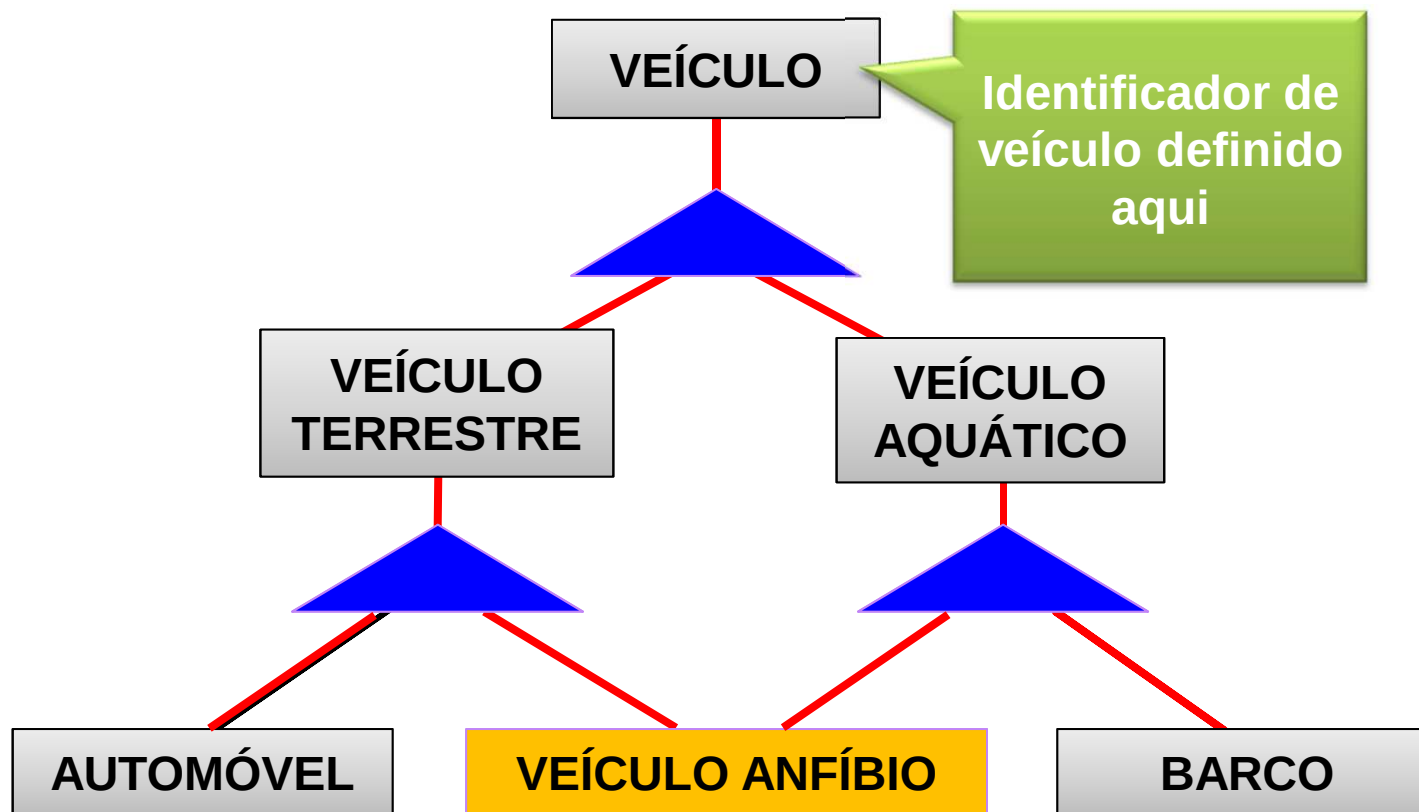
GRT1605

Generalização / Especialização



Generalização / Especialização (recursão)

A árvore de herança deve ter uma única entidade, que define o identificador.

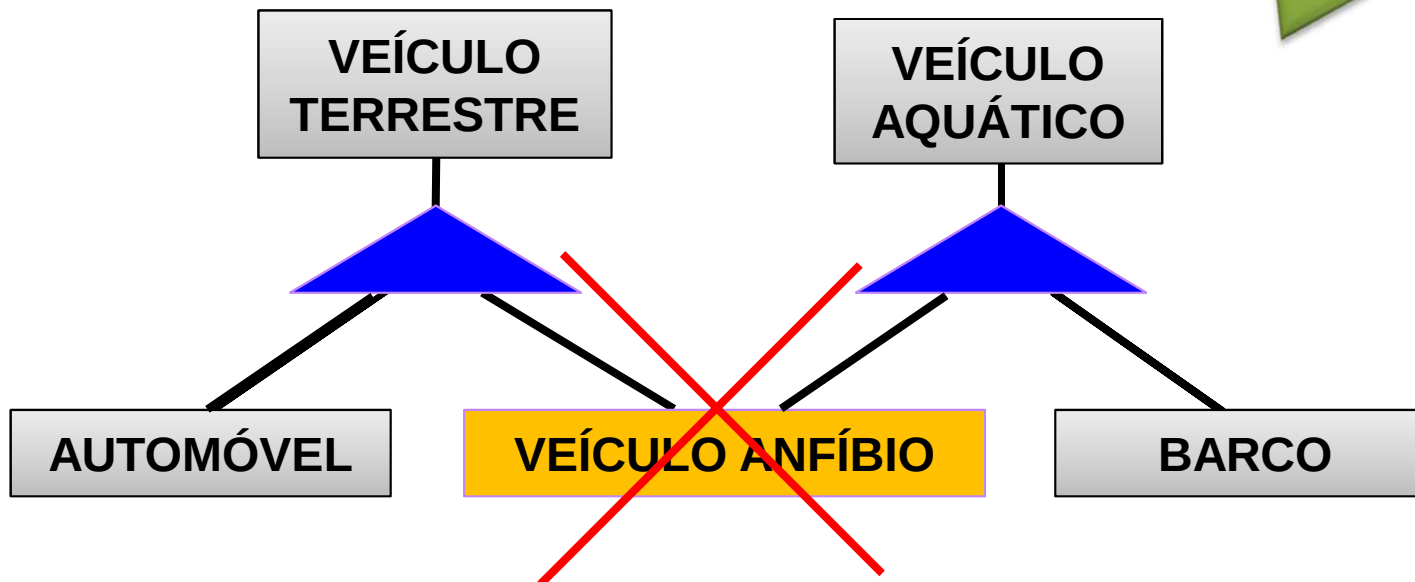


Generalização / Especialização (recursão)

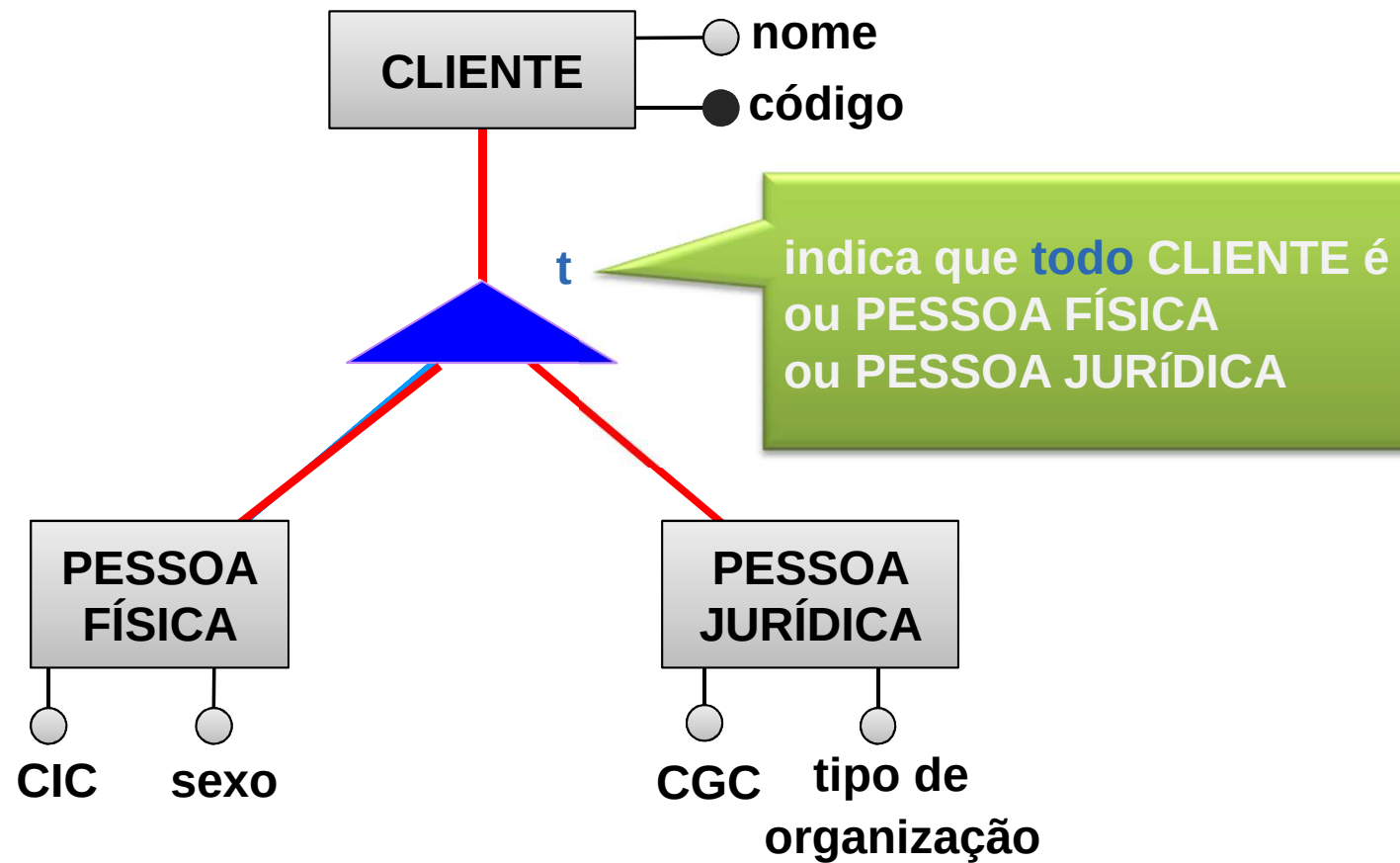
Caso proibido:

- Herança de múltiplos identificadores.

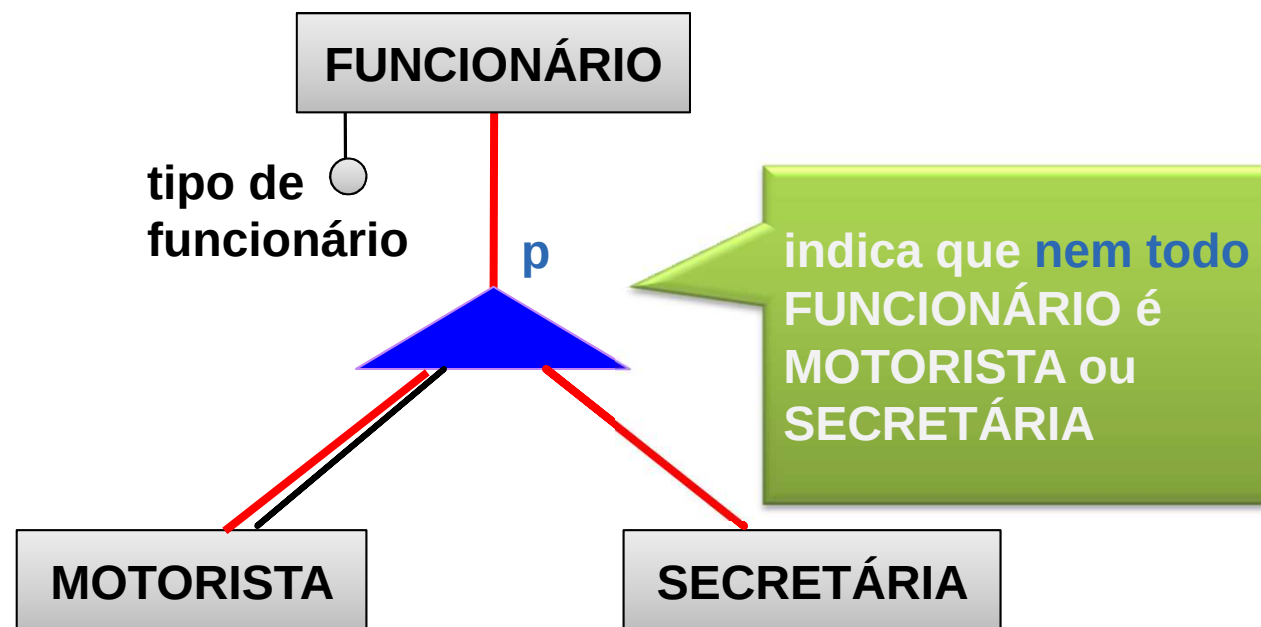
Qual seria o
identificador
desta entidade?



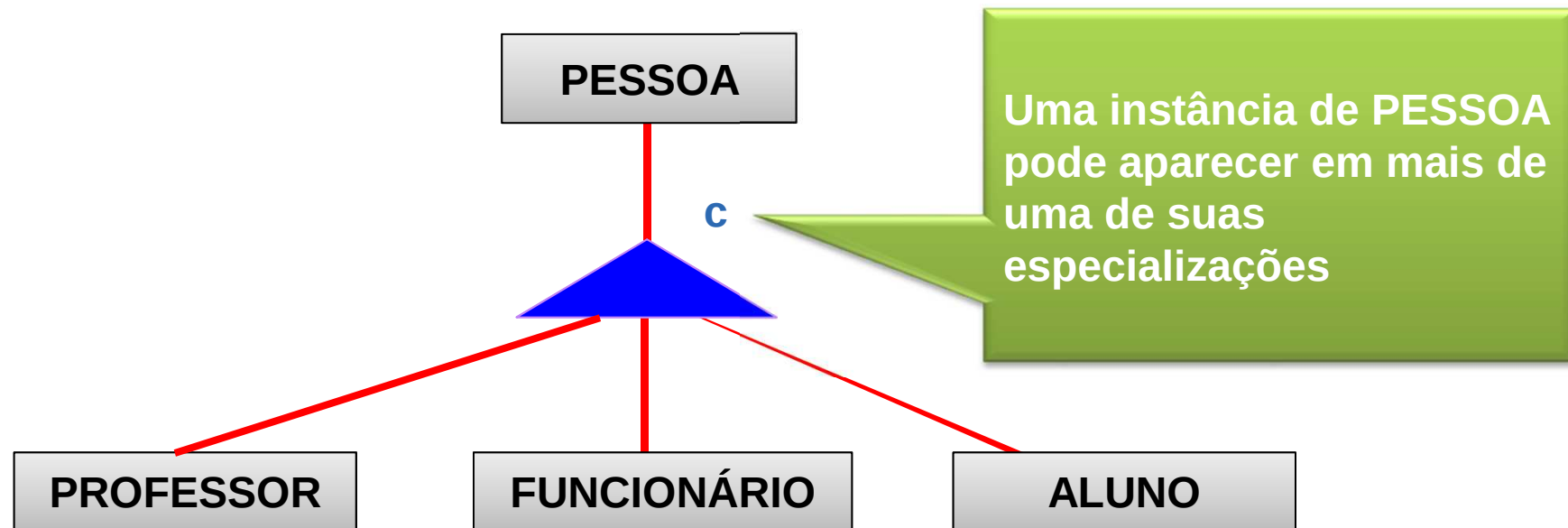
Especialização Total



Especialização Parcial



Especialização não exclusiva (compartilhada)



Linguagens de Banco de dados

SQL

Os comandos em **SQL** são classificados em tres grupos :

DDL – Definition Data Language

Comandos usados para criar e alterar tabelas, comandos que definem a estrutura dos dados.

DML – Data Manipulation Language

Comandos utilizados para extrair informações das tabelas, manipula os dados existentes.

DCL – Data Control Language

Comandos usados em sistemas multiusuário para definir os privilégios de acesso aos dados de cada usuário, usado para implementar a segurança e privacidade dos dados.

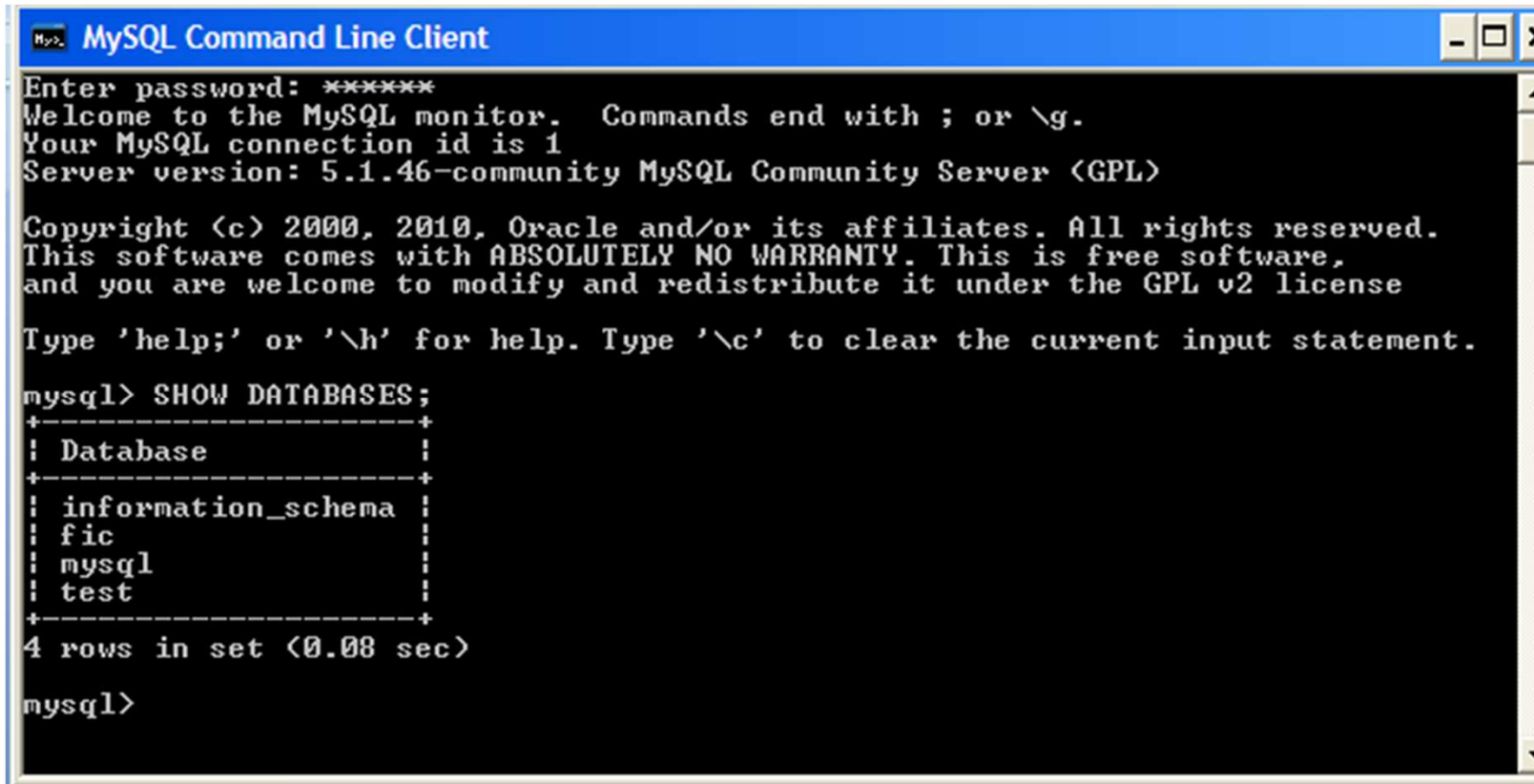
SQL

- Mantida pelo comitê **ANSI (American National Standards Institute)** Data Language
- Primeiro padrão em 1986
- Primeira atualização em 1992, conhecida como **SQL92** ou **SQL2**
- Última atualização em 1999, conhecida como **SQL99** ou **SQL3**
- **ISO (International Standards Organization)** assumiu também como padrão esta última versão.
- Alguns produtores de softwares lançam com frequência versões próprias do SQL, adicionando novos comandos e funcionalidades, estas versões são chamadas de “**DIALETOS**”
- Alguns **DIALETOS** trazem comandos como :
 (IF THEN ELSE....)
 (WHILE)
- Para alguns, isto descaracteriza a linguagem **SQL** que foi desenvolvida para manipulação de dados.

Mysql Command Line Client

Para iniciar, digite sua senha [enter]

Para verificar a existência de bancos, digite :
mysql> **SHOW DATABASES;** [enter]



```
MySQL Command Line Client
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 1
Server version: 5.1.46-community MySQL Community Server (GPL)

Copyright (c) 2000, 2010, Oracle and/or its affiliates. All rights reserved.
This software comes with ABSOLUTELY NO WARRANTY. This is free software,
and you are welcome to modify and redistribute it under the GPL v2 license

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| fic       |
| mysql     |
| test      |
+-----+
4 rows in set (0.08 sec)

mysql>
```

Mysql Command Line Client

Para criar um novo Banco de Dados, digite :

```
mysql> CREATE DATABASE FicTeste; [enter]  ou  
mysql> CREATE DATABASE if not exists FicTeste; [enter]
```

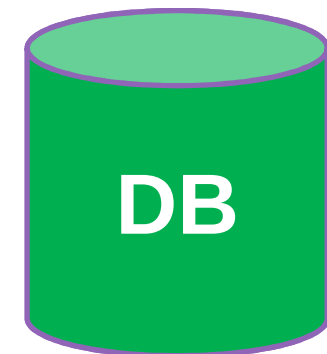
Para o nome dos bancos de dados, você pode usar até **64 caracteres**, com **letras** , **números**, **traços** e **underlines**. Não é aconselhável o uso de **acentos** e **cedilhas**. É proibido o uso de **barras** e **pontos**.

Para Excluir um Banco de Dados, digite :

```
mysql> DROP DATABASE FicTeste; [enter]  ou  
mysql> DROP DATABASE if exists FicTeste; [enter]
```

Para Selecionar um Banco de Dados, digite :

```
mysql> USE FicTeste; [enter]
```



Mysql Command Line Client

Para listar as tabelas de um Banco de Dados, digite :

mysql> **SHOW TABLES;** [enter]

Para criar uma tabela nova no Banco de Dados, digite :

mysql> **CREATE TABLE TB_ALUNOS (MATR CHAR(5), NOME CHAR (40));** [enter]

Para visualizar a estrutura da tabela, digite :

mysql> **DESCRIBE TB_ALUNOS;** [enter]

```

MySQL Command Line Client
mysql> USE TEST
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_test |
+-----+
| tb_alunos      |
+-----+
1 row in set (0.05 sec)

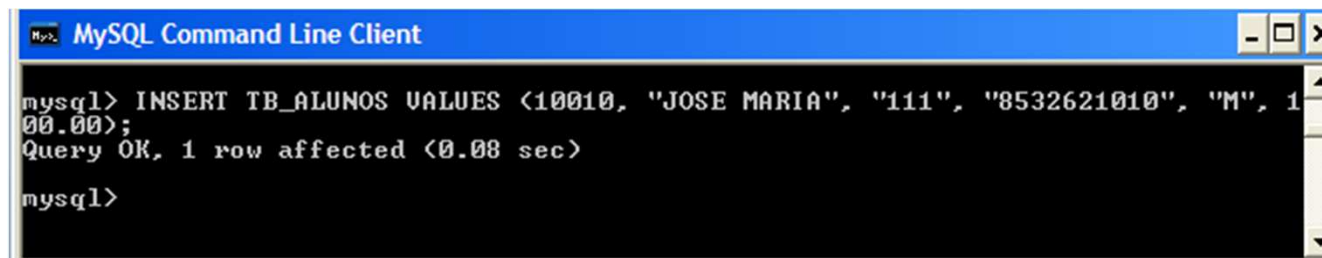
mysql> DESCRIBE TB_ALUNOS;
+-----+-----+-----+-----+-----+-----+
| Field      | Type      | Null | Key | Default | Extra      |
+-----+-----+-----+-----+-----+-----+
| nu_matricula | int(11)   | NO   | PRI | NULL    | auto_increment |
| nm_aluno     | varchar(45) | YES  |     | NULL    |              |
| nu_cpf       | varchar(11) | YES  |     | NULL    |              |
| nu_telefone  | varchar(10) | YES  |     | NULL    |              |
| cd_sexo      | varchar(1) | YES  |     | NULL    |              |
| vl_mensal    | float(10,2) | YES  |     | NULL    |              |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.03 sec)
  
```



Mysql Command Line Client

Para Inserir dados na tabela, digite :

```
mysql> INSERT TB_ALUNOS VALUES (10010, "JOSE MARIA",  
"111", "8532621010", "M", 100.00); [enter]
```

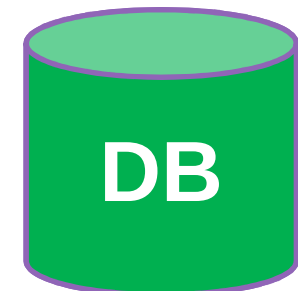


```
MySQL Command Line Client  
mysql> INSERT TB_ALUNOS VALUES (10010, "JOSE MARIA", "111", "8532621010", "M", 100.00);  
Query OK, 1 row affected (0.08 sec)  
mysql>
```

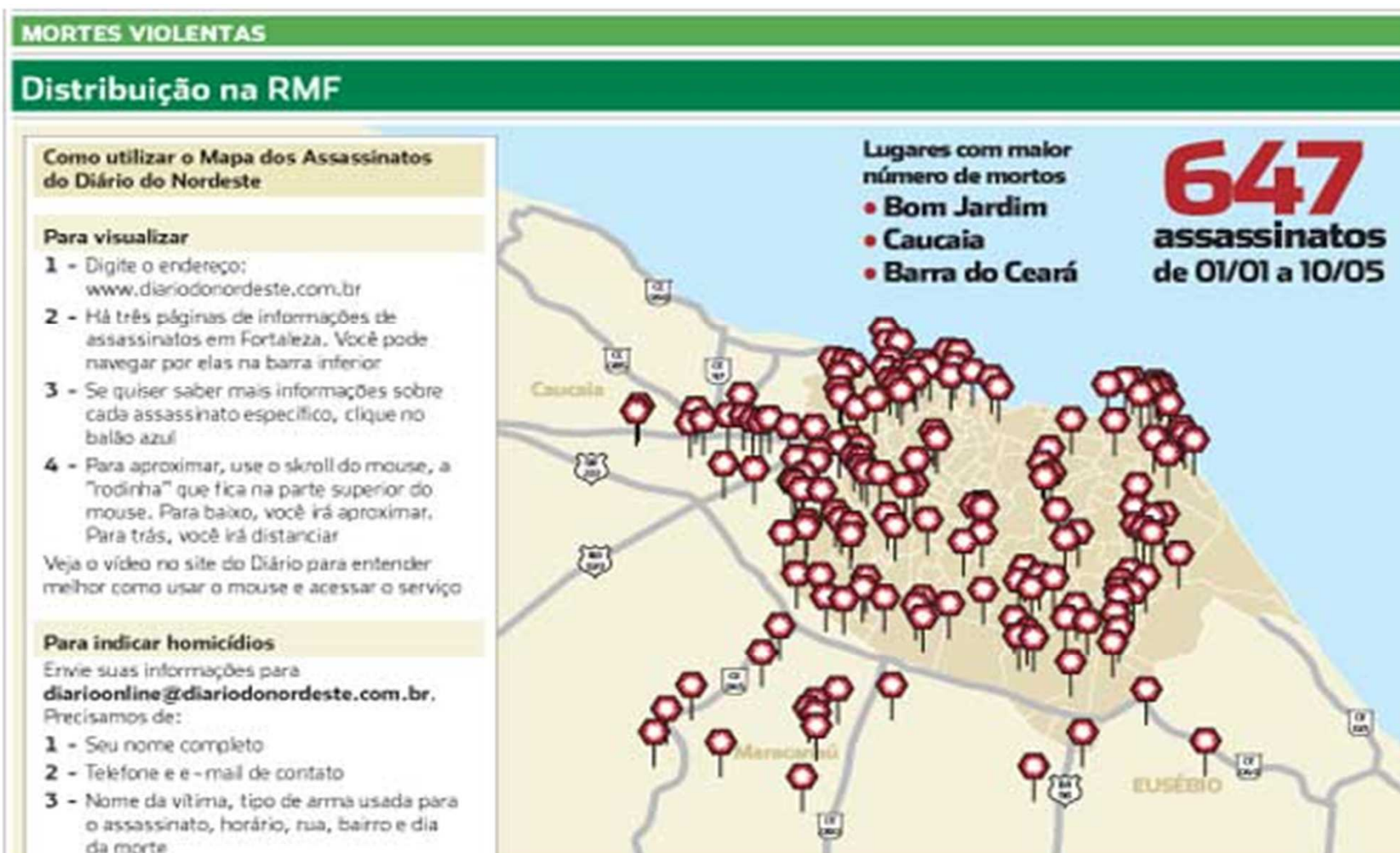
Tipo de dados : CHAR e VARCHAR

CHAR(80) – Sempre ocupará 80 bytes

VARCHAR(80) – Ocupará no máximo 80 bytes, este tipo e de tamanho variável.



Modelo de Entidades e Relacionamentos

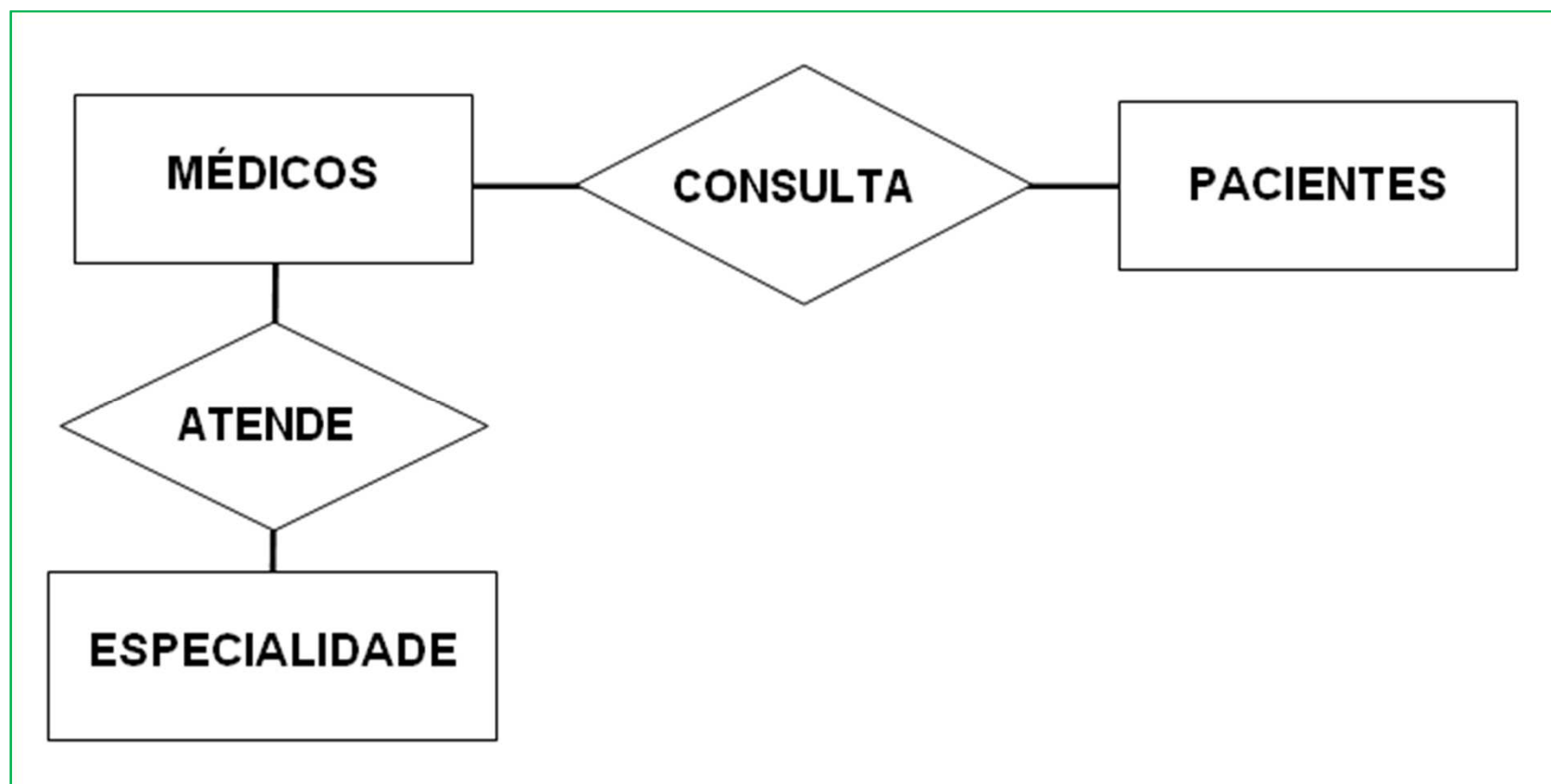


Edição Diário do Nordeste 11/maio/2010

[clique aqui para fechar](#)

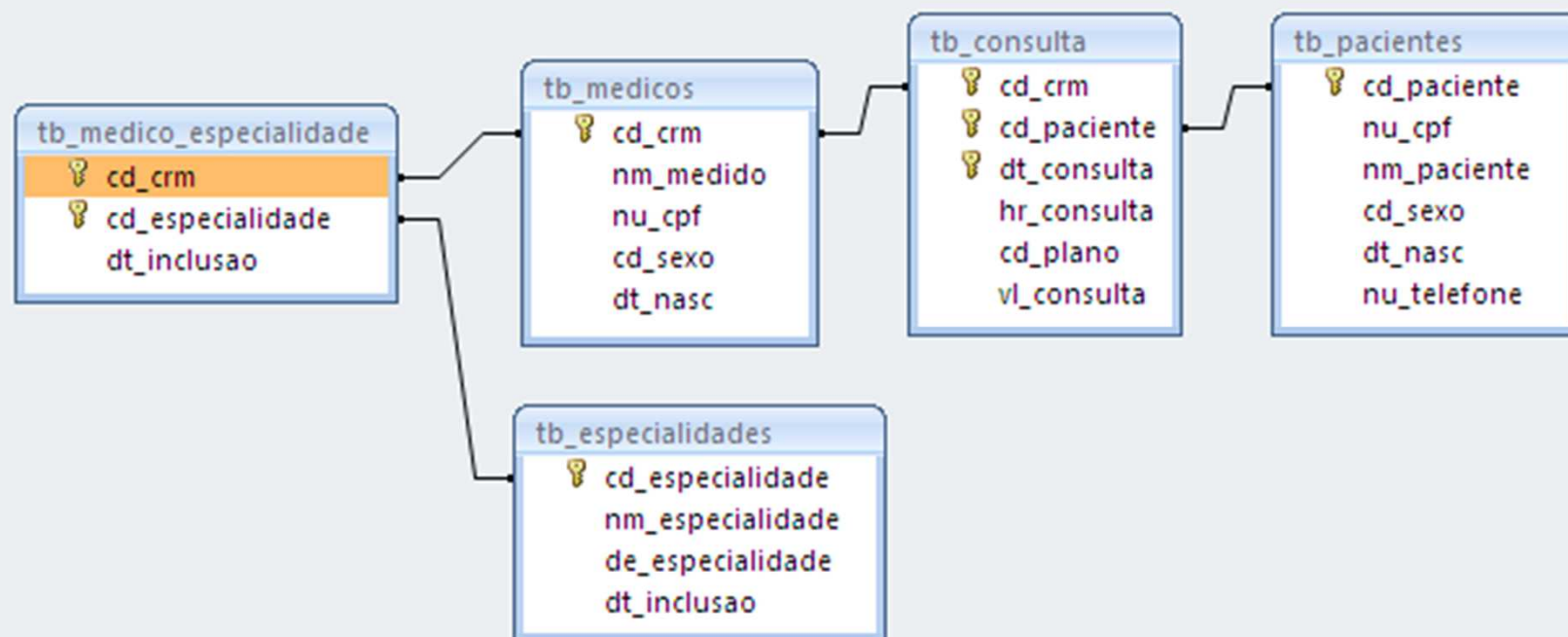


Modelo de Entidades e Relacionamentos



Modelo de Entidades e Relacionamentos

Modelo Lógico – Clínica Médica



Tipo de dados - Mysql

1. Números

Tipo de Campo	Tamanho de Armazenamento
TINYINT	1 byte
SMALLINT	2 bytes
MEDIUMINT	3 bytes
INT	4 bytes
INTEGER	4 bytes
BIGINT	8 bytes
FLOAT(X)	4 ou 8 bytes
FLOAT	4 bytes
DOUBLE	8 bytes
DOUBLE PRECISION	8 bytes
REAL	8 bytes
DECIMAL(M,D)	M+2 bytes se $D > 0$, M+1 bytes se $D = 0$
NUMERIC(M,D)	M+2 bytes se $D > 0$, M+1 bytes se $D = 0$

1 Tipos numéricos:

Os dados numéricos podem dividir-se em dois grandes grupos, os que tem vírgula flutuante (com decimais) e os que não tem.

TinyInt: é um número inteiro com ou sem sinal. Com sinal a margem de valores válidos é de -128 até 127. Sem sinal, a margem de valores é de 0 até 255

Bit ou Bool: um número inteiro que pode ser 0 ou 1.

SmallInt: número inteiro com ou sem sinal. Com sinal a margem de valores válidos é de -32768 até 32767. Sem sinal, a margem de valores é de 0 até 65535.

MediumInt: número inteiro com ou sem sinal. Com sinal a margem de valores válidos é de -8.388.608 até 8.388.607. Sem sinal, a margem de valores é de 0 até 16777215.

Integer, Int: número inteiro com ou sem sinal. Com sinal a margem de valores válidos é de -2147483648 até 2147483647. Sem sinal, a margem de valores é de 0 até 429.496.295

BigInt: número inteiro com ou sem sinal. Com sinal a margem de valores válidos é de -9.223.372.036.854.775.808 até 9.223.372.036.854.775.807. Sem sinal, a margem de valores é de 0 até 18.446.744.073.709.551.615.

Float: número pequeno com vírgula flutuante de precisão simples. Os valores válidos vão desde -3.402823466E+38 até -1.175494351E-38,0

xReal, Double: número com vírgula flutuante de dupla precisão. Os valores permitidos vão desde -1.7976931348623157E+308 até -2.2250738585072014E-308,

Tipo de dados - Mysql

2. Data

Tipo de Campo	Tamanho de Armazenamento
DATE	3 bytes
DATETIME	8 bytes
TIMESTAMP	4 bytes
TIME	3 bytes
YEAR	1 byte

Date: tipo data, armazena uma data. A margem de valores vai desde o 1 de Janeiro de 1001 ao 31 de dezembro de 9999. O formato de armazenamento é de ano-mes-dia.

DateTime: Combinação de data e hora. A margem de valores vai desde o 1 de Janeiro de 1001 às 0 horas, 0 minutos e 0 segundos até 31 de Dezembro de 9999 às 23 horas, 59 minutos e 59 segundos. O formato de armazenamento é de ano-mes-dia horas:minutos:segundos

TimeStamp: Combinação de data e hora. A margem vai desde o 1 de Janeiro de 1970 ao ano 2037. O formato de armazenamento depende do tamanho do campo:

Time: armazena uma hora. A margem de horas vai desde -838 horas, 59 minutos e 59 segundos. O formato de armazenamento é '**HH:MM:SS**'.

Year: armazena um ano. A margem de valores permitidos vai desde o ano 1901 ao ano 2155. O campo pode ter tamanho dois ou tamanho 4 dependendo de se queremos armazenar o ano com dois ou quatro algarismos.

Tipo de dados - Mysql



3. Strings

Char(n): armazena uma cadeia de tamanho fixo. A cadeia poderá conter de 0 até 255 caracteres.

VarChar(n): armazena uma cadeia de tamanho variável. A cadeia poderá conter de 0 até 255 caracteres. Dentro dos tipos de cadeia pode-se distinguir dois subtipos, os tipo **Text** e os tipo **Blob** (Binary Large Object) A diferença entre um tipo e outro é o tratamento que recebem na hora de ordená-los e compará-los. No tipo **text** ordena-se sem ter importância as maiúsculas e as minúsculas e no tipo **blob** ordena-se levando em conta as maiúsculas e minúsculas.

TinyText e TinyBlob: Coluna com um tamanho máximo de 255 caracteres.

Blob e Text: um texto com um máximo de 65535 caracteres.

MediumBlob e MediumText: um texto com um máximo de 16.777.215 caracteres.

LongBlob e LongText: um texto com um máximo de caracteres 4.294.967.295.

Enum: campo que pode ter um único valor de uma lista especificada. O tipo Enum aceita até 65535 valores diferentes.

Set: um campo que pode conter nenhum, um ou vários valores de uma lista. A lista pode ter um máximo de 64 valores.

Tipo de dados - Mysql

3. Strings

Tipo de campo	Tamanho de Armazenamento
CHAR(n)	n bytes
VARCHAR(n)	n +1 bytes
TINYBLOB, TINYTEXT	Tamanho+1 bytes
BLOB, TEXT	Tamanho+2 bytes
MEDIUMBLOB, MEDIUMTEXT	Tamanho +3 bytes
LOB, LONGTEXT	Tamanho+4 bytes
ENUM('value1','value2',...)	Dependendo da quantidade de valores
SET('value1','value2',...)	1, 2, 3, 4 ou 8 bytes, dependendo da quantidade de valores

Mysql Command Line Client

Para alterar o tipo de campos de uma tabela, digite :

```
mysql> ALTER TABLE tabela MODIFY campo DATE; [enter]
```

Para deletar um mais registros de uma tabela, digite :

```
mysql> DELETE FROM tabela WHERE nu_matric = 10001; [enter]
```

```
mysql> DELETE FROM tabela WHERE cd_uf = "CE"; [enter]
```

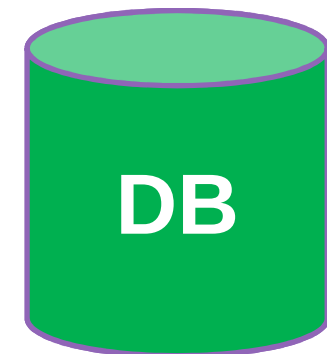
Para a atualizar os dados de uma tabela, digite :

```
mysql> UPDATE tabela SET cd_sexo = "M"; [enter]
```

```
mysql> UPDATE tabela SET vl_salario = vl_salario * 1.12; [enter]
```

Para excluir todos os dados de uma tabela, digite :

```
mysql> TRUNCATE TABLE tabela; [enter]
```



Linguagem de Banco de dados

Finalidade:

1. Garantir a especificação de um esquema de BD
2. Permitir consultas e atualizações sobre o BD

Linguagem de Banco de dados

DDL: Data Description Language

1. Utilizada para especificar o esquema de um BD

Exemplo (SQL)

```
create table Empregado  
(matr integer not null,  
nome varchar(35),  
salário real,  
primary key(matr))
```

Linguagem de Banco de dados

DML: Data Manipulation Language

1. Utilizada para permitir consultas em um BD

Exemplo (SQL)

```
select nome  
from Empregado  
where salario > 7000
```

Linguagem de Banco de dados

DML: Data Manipulation Language

2. Utilizada para permitir inserções em uma tabela

Exemplo (SQL)

```
insert into Empregado  
values(123, “Bárbara”, 5000.00)
```

Linguagem de Banco de dados

DML: Data Manipulation Language

3. Utilizada para permitir remoções em uma tabela

Exemplo (SQL)

```
delete from Empregado  
where matr=14
```

DML: Data Manipulation Language

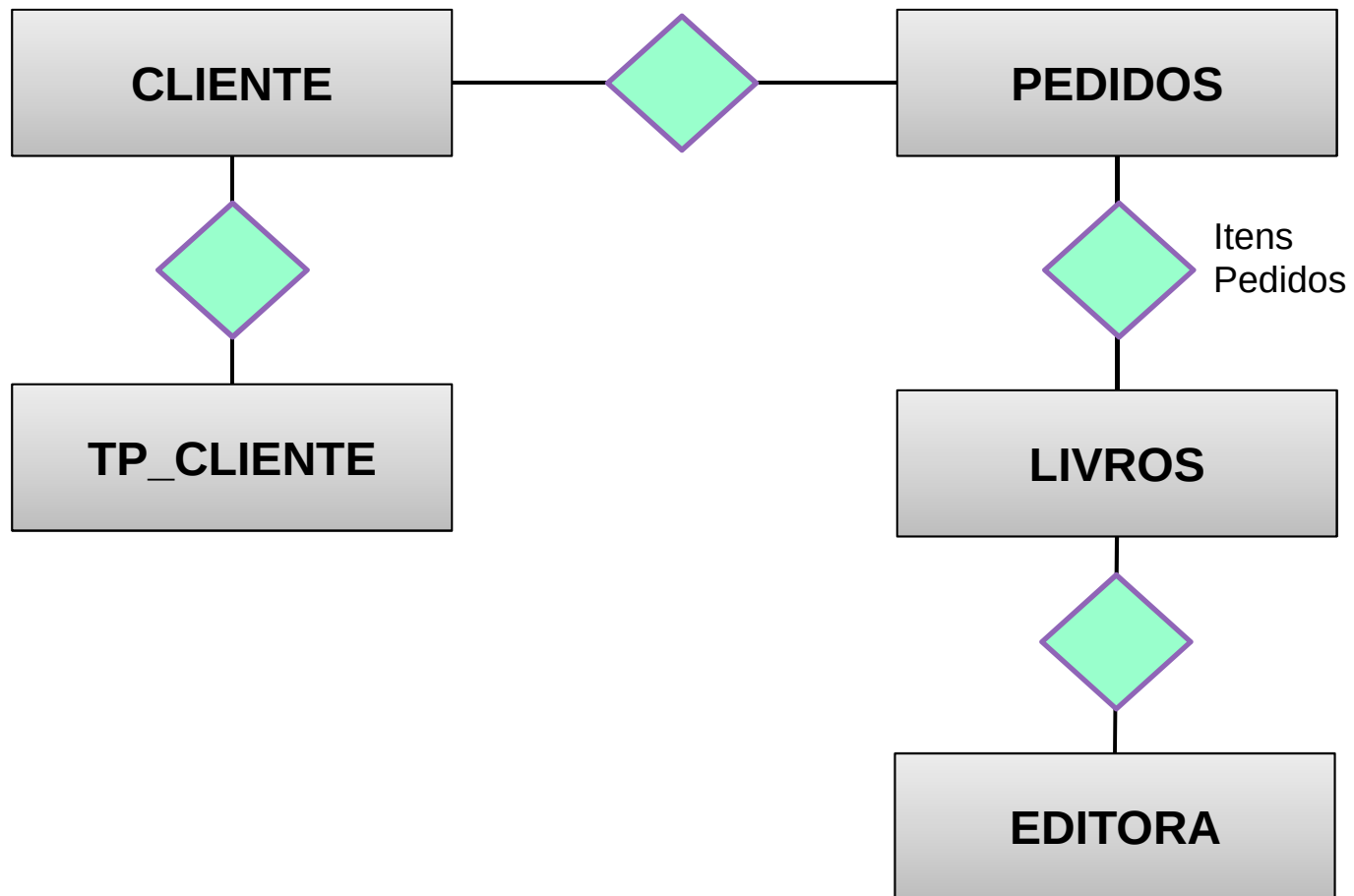
4. Utilizada para permitir atualizações de valores de atributos de uma tabela

Exemplo (SQL)

```
update Empregado set salário=salário*1.15  
where salário<1500.00
```

Sentenças SQL

MER – Venda de Livros



Mo – Venda de Livros

Nome	Tipo de Dados	Nulo	Padrão	Extras
Índice Primário	CD_CLIENTE			unique
CD_CLIENTE	varchar(5)	Não		
NM_CLIENTE	varchar(31)	Não		
TP_CLIENTE	tinyint(1)	Não		
CD_SEXO	varchar(1)	Não		

Nome	Tipo de Dados	Nulo	Padrão	Extras
Índice Primário	TP_CLIENTE			unique
TP_CLIENTE	tinyint(1)	Não		
DE_TIPO_CL...	varchar(6)	Não		

Nome	Tipo de Dados	Nulo	Padrão	Extras
Índice Primário	CD_EDITORA			unique
CD_EDITORA	varchar(4)	Não		
NM_EDITORA	varchar(27)	Não		

Nome	Tipo de Dados	Nulo	Padrão	Extras
Índice Primário	NU_PEDIDO,CD...			unique
NU_PEDIDO	mediumint(6)	Não		
CD_LIVRO	varchar(13)	Não		
QT_LIVROS	int(5)	Não	0	

Nome	Tipo de Dados	Nulo	Padrão	Extras
Índice Primário	ISBN			unique
ISBN	varchar(13)	Não		
NM_TITULO	varchar(58)	Não		
NU_EDICAO	varchar(2)	Não		
ORIGEM	varchar(70)	Não		
CD_EDITORA	varchar(5)	Sim	<Nulo>	
DT_ESTOQUE	date	Sim	<Nulo>	
VL_VENDA	float(10,2)	Sim	<Nulo>	

Nome	Tipo de Dados	Nulo	Padrão	Extras
Índice Primário	CD_CLIENTE,N...			unique
CD_CLIENTE	varchar(5)	Não		
NU_PEDIDO	mediumint(6)	Não		
DT_PEDIDO	date	Sim	<Nulo>	

Exemplos SQL

```
select * from tb_aluno
```

```
select matric, nome, sexo from tb_aluno
```

```
select matric, nome, sexo from tb_aluno where sexo = "F"
```

```
select matric, nome, sexo from tb_aluno where sexo = "F"  
and nome like "Maria%"
```

```
select matric, nome, sexo from tb_aluno where sexo = "F"  
and nome like "Maria%" order by nome desc
```

```
select matric, nome, sexo from tb_aluno where matric =  
10001 or matric = 10002 order by nome
```

Exemplos SQL

select count(*) from tb_aluno where matric like “2010%”

select sum(vl_pago) from tb_aluno where sexo = “F”

select max(nota_av1) from tb_aluno where sexo = “M”

select cd_curso, count(*) from tb_aluno group by cd_curso

**select cd_curso, count(*) from tb_aluno group by cd_curso
having count(*) = 1**

select matric, nome, sexo from tb_aluno limit 100

O que vai acontecer ?

```
SELECT Count(CódigoPostal) AS TCP FROM Clientes;
```

```
SELECT CategoriaID, Sum(UnidadesNoEstoque) FROM Produtos  
GROUP BY CategoriaID  
HAVING Sum(UnidadesNoEstoque) > 100 AND CategoriaID LIKE  
"BOS*";
```

```
SELECT Sobrenome, Nome FROM Funcionários ORDER BY  
Sobrenome ASC;
```

```
SELECT Sobrenome, Salário FROM Funcionários ORDER BY  
Salário DESC, Sobrenome;
```

```
SELECT Item, Sum(Unidades) AS Tbc FROM ItensEmEstoque  
GROUP BY Item, Cor;
```

```
UPDATE Pedidos SET ValorPedido = ValorPedido * 1.1,  
Frete = Frete * 1.03  
WHERE PaísEmbarque = 'RU';
```

O que vai acontecer ?

```
SELECT * FROM Produtos WHERE PreçoUnit > ANY  
(SELECT PreçoUnit FROM PedidoDetalhes WHERE Desconto >= .25);
```

```
SELECT * FROM Produtos WHERE PreçoUnit > ALL  
(SELECT PreçoUnit FROM PedidoDetalhes WHERE Desconto >= .25);
```

```
SELECT * FROM Produtos WHERE ProdutoID IN  
(SELECT ProdutoID FROM PedidoDetalhes WHERE Desconto >= .25);
```

```
SELECT * FROM Produtos WHERE ProdutoID NOT IN  
(SELECT ProdutoID FROM PedidoDetalhes WHERE Desconto >= .25);
```

```
SELECT Sobrenome, Nome, Titulo, Salario FROM Funcionarios AS T1  
WHERE Salário >= (SELECT Avg(Salário) FROM Funcionarios  
WHERE T1.Título = Funcionarios.Título)  
Order by Titulo;
```

O que vai acontecer ?

```
SELECT Sobrenome, Nome, Título, Salário FROM Funcionários  
WHERE Título LIKE "*Repr Vendas*" AND Salário > ALL  
(SELECT Salário FROM Funcionários WHERE (Título LIKE "*Gerente*")  
OR (Título LIKE "*Diretor*"));
```

```
SELECT NomeProduto, PreçoUnit FROM Produtos  
WHERE PreçoUnit = (SELECT PreçoUnit FROM Produtos  
WHERE NomeProduto = "Licor de Cacau");
```

```
SELECT * From FUNCIONARIO WHERE MATRIC_FUNC > 10)  
UNION  
SELECT * From FUNCIONARIO WHERE DEPARTAM_FUNC > 10);
```

O que vai acontecer ?

```
SELECT Sobrenome, Nome, Título, Salário FROM Funcionários  
WHERE Título LIKE "*Repr Vendas*" AND Salário > ALL  
(SELECT Salário FROM Funcionários WHERE (Título LIKE "*Gerente*")  
OR (Título LIKE "*Diretor*"));
```

```
SELECT NomeProduto, PreçoUnit FROM Produtos  
WHERE PreçoUnit = (SELECT PreçoUnit FROM Produtos  
WHERE NomeProduto = "Licor de Cacau");
```

```
SELECT * From FUNCIONARIO WHERE MATRIC_FUNC > 10)  
UNION  
SELECT * From FUNCIONARIO WHERE DEPARTAM_FUNC > 10);
```

INNER JOIN e LEFT JOIN

LOCALHOST\SQL...QLQuery1.sql* Summary

```
select e.nome, p.valor as pagamento  
from empregados as e LEFT JOIN pagamentos as p  
on e.codigo_empregado = p.codigo_empregado
```



Results



Messages

	nome	pagamento
1	Luis	100.00
2	Luis	200.00
3	Marina	NULL
4	Letícia	300.00
5	Gustavo	NULL
6	Mateus	400.00
7	Mateus	500.00

LOCALHOST\SQL...QLQuery1.sql* Summary

```
select e.nome, p.valor as pagamento  
from empregados as e inner JOIN pagamentos as p  
on e.codigo_empregado = p.codigo_empregado
```



Results



Messages

	nome	pagamento
1	Luis	100.00
2	Luis	200.00
3	Letícia	300.00
4	Mateus	400.00
5	Mateus	500.00

INNER JOIN e LEFT JOIN

```
select c.cd_cliente, nm_cliente, c.tp_cliente, de_tipo_cliente,  
       p.nu_pedido, p.dt_pedido from tb_clientes c  
       inner join tb_tipo_cliente t on c.tp_cliente = t.tp_cliente  
       inner join tb_pedidos p    on c.cd_cliente = p.cd_cliente  
                                order by c.cd_cliente
```

```
select m.nome, c.modelo from  marcas as m cross join  carros as c
```

```
select m.nome, c.modelo from  marcas as m inner join  carros as c  
on c.marca = m.marca
```

```
select m.nome, c.modelo from  marcas as m left join    carros as c  
on c.marca = m.marca
```

```
select m.nome, c.modelo from  marcas as m right join   carros as c  
on c.marca = m.marca
```

```
select m.nome, c.modelo from  marcas as m full outer join carros as c  
on c.marca = m.marca
```


FUNÇÕES

FUNÇÃO	DESCRIÇÃO
CURRENT_DAY	Retorna a Data corrente do relógio do sistema
CURRENT_TIME	Retorna a Hora corrente do relógio do sistema
CURRENT_TIMESTAMP	Retorna a Data e Hora do relógio do sistema
CURRENT_USER	Retorna o usuário logado no Banco de dados
CHAR_LENGTH(expressão)	Retorna o número de caracteres de uma expressão
EXTRACT (parte from expressão)	Extraí uma parte de uma data (YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, TIMEZONE_HOUR, TIMEZONE_MINUTE)
POSITION (parte IN expressão)	Retorna a posição inicial da string na expressão
SUBSTRING(expressão,início, fim)	Extraí parte de uma sequência de caracteres
LOWER(expressão)	Converte as letras em caracteres minúsculos
UPPER(expressão)	Converte as letras em caracteres maiúsculas
TRIM(expressão)	Retira caracteres brancos do início e fim de uma string

CHAVE PRIMÁRIA

Valor do registro que jamais poderá se repetir na mesma coluna, ou seja, servirá para identificar cada entrada da tabela de maneira **unívoca**.

create table Empregado
(matr integer not null, nome varchar(35), salário real,
primary key(matr))

MATRÍCULA	NOME	SALÁRIO
10001	FRANCISCO JOSÉ	2500,00
10005	JOSÉ SANTANA	1500,00
10200	ARI ARAUJO	510,00
10300	RENATO JOSÉ	600,00
10500	WILLIAM SENA	1200,00

CHAVE PRIMÁRIA COMPOSTA

Em alguns casos para identificar um registro de forma única, precisaremos unir dois ou mais campos da tabela, neste caso é necessário criar um **CHAVE COMPOSTA**.

```
create table TB_TELEFONE  
(DDD integer not null, TERMINAL integer not null,  
  nm_cliente varchar(35), DT_VENC date, CONTA_MES real,  
  primary key(DDD, TERMINAL))
```

DDD	TERMINAL	NM_CLIENTE	DT_VENC	CONTA_MES
85	34582500	FRANCISCO JOSÉ	15/09/2010	2500,00
85	34582000	JOSÉ SANTANA	20/09/2010	1500,00
85	34593000	ARI ARAUJO	30/08/2010	510,00
88	34582000	RENATO JOSÉ	15/08/2010	600,00
88	34582500	WILLIAM SENA	16/08/2010	1200,00

TABELAS e ÍNDICES

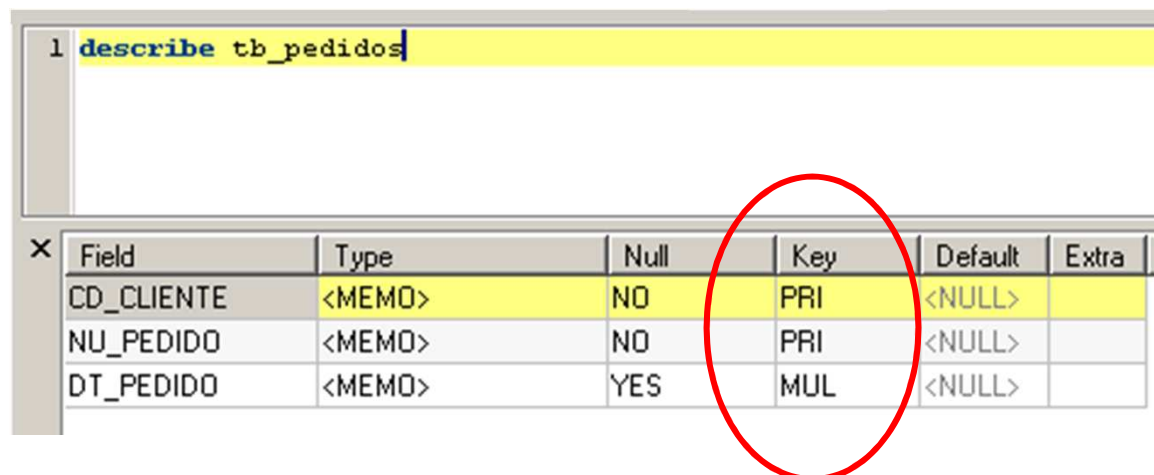
O termo **CHAVE** ou **KEY** é sinônimo de **ÍNDICE**, que é uma ferramenta interna do SQL.

Um **ÍNDICE** é criado para agilizar as operações de acesso aos dados .

Para criar um **ÍNDICE** podemos usar :

ALTER TABLE TB_TELEFONE
ADD INDEX VENCIMENTO (DT_VENC);

Este comando criará um **ÍNDICE** ou **KEY** do tipo **MUL**, ou múltiplo.



```
1 describe tb_pedidos
```

Field	Type	Null	Key	Default	Extra
CD_CLIENTE	<MEMO>	NO	PRI	<NULL>	
NU_PEDIDO	<MEMO>	NO	PRI	<NULL>	
DT_PEDIDO	<MEMO>	YES	MUL	<NULL>	

TABELAS e ÍNDICES

Para criar um ÍNDICE podemos usar também :

```
CREATE INDEX VENCIMENTO ON TB_TELEFONE (DT_VENC)
```

No MYSQL uma tabela pode ter até 16 índices.
Cada software tem sua limitação.

Porque devemos usar ÍNDICES ?



**ESTRUTURA
DE DADOS**

TABELAS e ÍNDICES

O SELECT EXECUTA UMA CONSULTA SEQUENCIAL,
VELOCIDADE É INPERCEPTÍVEL COM PEQUENAS TABELAS.

FISICAMENTE, UM ÍNDICE É UMA PEQUENA TABELA,
COMPOSTA DE DOIS CAMPOS :

- O PRIMEIRO É GERADO PELO SQL, POSIÇÃO DA LISTA
- O SEGUNDO É A POSIÇÃO DOS DADOS

DDD	TERMINAL	NM_CLIENTE	DT_VENC	CONTA_MES
85	34582500	FRANCISCO JOSÉ	15/09/2010	2500,00
85	34582000	JOSÉ SANTANA	20/09/2010	1500,00
85	34593000	ARI ARAUJO	30/08/2010	510,00
88	34582000	RENATO JOSÉ	15/08/2010	600,00
88	34582500	WILLIAM SENA	16/08/2010	1200,00

ORDEM	POSIÇÃO
1	4
2	5
3	1
4	2
5	3

VALORES PRÉ-DEFINIDOS OU DEFAULT

O valor default para campos CHAR ou VARCHAR é **NULL**
Para campos NUMÉRICOS não há valor pré-definido.

Quando definimos um campo como NOT NULL, sempre deverá ser preenchido este campos.

```
create table Empregado  
(matr integer not null,  
  nome varchar(35), salário real not null DEFAULT 0,  
  primary key(matr))
```

EXCLUINDO TABELAS

Para excluirmos todos os arquivos de uma tabela, usamos o comando :

DROP TABLE **tabela**

Se desejar excluir apenas o ÍNDICE, use :

DROP INDEX **VENCIMENTO** **ON TB_MOVIMENTOS**

Ou

ALTER TABLE TB_MOVIMENTOS DROP INDEX VENCIMENTO

Para Excluir um Banco de Dados, digite :

mysql> **DROP DATABASE** **BANCO1**; [enter] ou

mysql> **DROP DATABASE** if exists **BANCO1**; [enter]

TABELAS - HOTEL

TB_CLIENTE
CD_CLIENTE, varchar(5)
NM_CLIENTE, varchar(50)
CD_SEXO, char(1)
DT_NASC, date

TB_HOSPEDAGEM
CD_CLIENTE, varchar(5)
NU_APTO, int(4)
DT_INICIO, date
DT_FIM, date

TB_TIPO_APTO
CD_TIPO_APTO, char(2)
NM_TIPO_APTO, varchar(30)

TB_CONSUMO
CD_CLIENTE, varchar(5)
CD_PRODUTO, varchar(5)
DT_CONSUMO, date
HR_CONSUMO, date
QT_CONSUMO, int(5)
VL_TOTAL, float(10,2)

TB_APTO
NU_APTO, int (4)
NM_APTO, varchar(30)
CD_TIPO, char(2)

TB_SERVICOS
CD_PRODUTO, varchar(5)
NM_PRODUTO, varchar(50)
DE_PRODUTO, varchar(100)
TP_PRODUTO, char(2)
VL_PRODUTO, float(10,2)

VIEWS

CREATE VIEW <view_name> AS <instrução_SELECT>

Exemplo:

**CREATE VIEW V_TAB1 AS
SELECT COD , NOME FROM TABELA1**

- Esta VIEW só retorna dois campos da TABELA1 (COD e NOME.
- Agora podemos usar a VIEW como uma tabela normal :
SELECT COD FROM V_TAB1 WHERE NOME = 'Mauro'

STORED PROCEDURE

- É um **conjunto de comandos**, ao qual é atribuído um nome. Este conjunto fica armazenado no Banco de Dados e **pode ser chamado** a qualquer momento tanto pelo SGBD quanto por um sistema.

DELIMITER &&

CREATE PROCEDURE *inserir_aluno*

(*IN* nu_matric INT, nm_aluno CHAR(45), cd_sexo CHAR(1))

BEGIN

REPLACE INTO tb_aluno VALUES (nu_matric, nm_aluno, cd_sexo);

END &&

CALL *inserir_aluno* (10001, "MARIA JOSÉ", "F");

TRIGGERS

- É um **conjunto de comandos**, ao qual é atribuído um nome. Este conjunto fica armazenado no Banco de Dados e **pode ser chamado** a qualquer momento tanto pelo SGBD quanto por um sistema.

```
CREATE TABLE test1(a1 INT);  
CREATE TABLE test2(a2 INT);  
CREATE TABLE test3(a3 INT NOT NULL AUTO_INCREMENT PRIMARY KEY);  
CREATE TABLE test4(a4 INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
b4 INT DEFAULT 0);
```

DELIMITER |

```
CREATE TRIGGER teste BEFORE INSERT ON test1  
FOR EACH ROW  
BEGIN  
    INSERT INTO test2 SET a2 = NEW.a1;  
    DELETE FROM test3 WHERE a3 = NEW.a1;  
    UPDATE test4 SET b4 = b4 + 1 WHERE a4 = NEW.a1;  
END;
```